

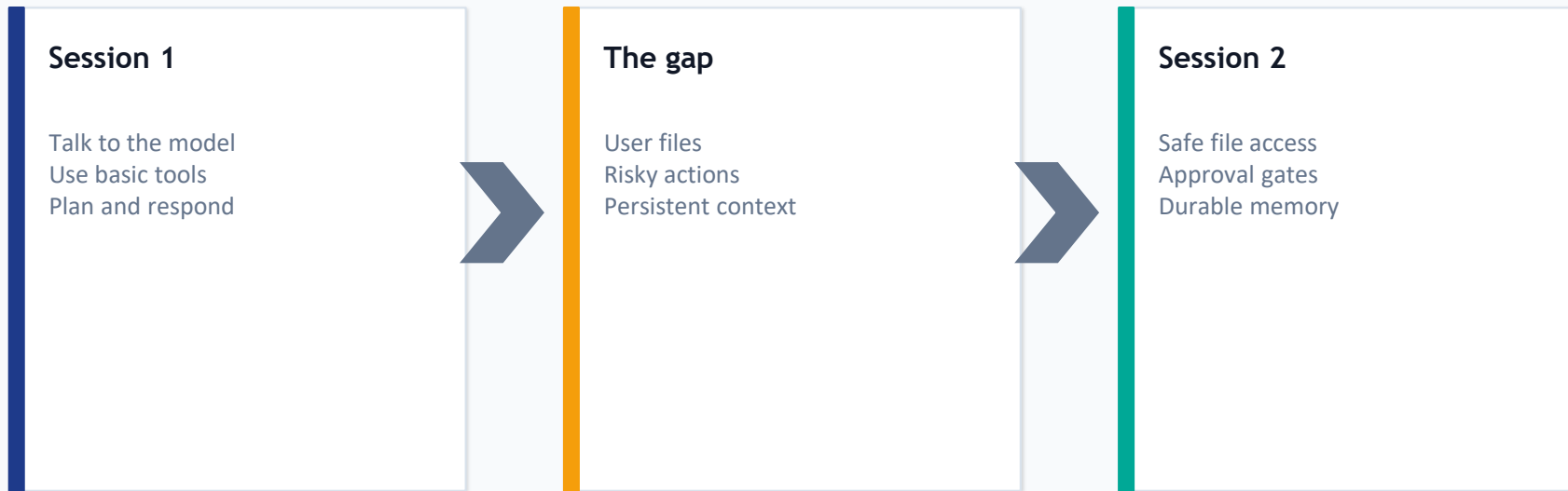
Working with your data, safely

Files, approvals, and memory for a finance advisor built with
Microsoft Agent Framework + Harness

**TRUST
BOUNDARY**

What changed since Session 1?

The agent can talk and use tools. Now it needs boundaries before touching user-owned data.



The three trust boundaries

A useful agent is not just smarter. It is bounded by the host application.

Files

Read portfolio data from an approved working folder. No arbitrary filesystem access.

Approvals

Pause before writes or simulated trades. Let the human decide before side effects.

Memory

Persist useful context deliberately. Separate local file memory from platform memory.

The model chooses among tools; C# owns the boundary.

Safe file access: trust the working folder

The assistant can inspect real portfolio data without receiving raw filesystem permission.

```
working/  
  portfolio.csv  
  memory.json  
  reports/  
    portfolio-summary.md
```

10

Plain C# sandbox
guard



11

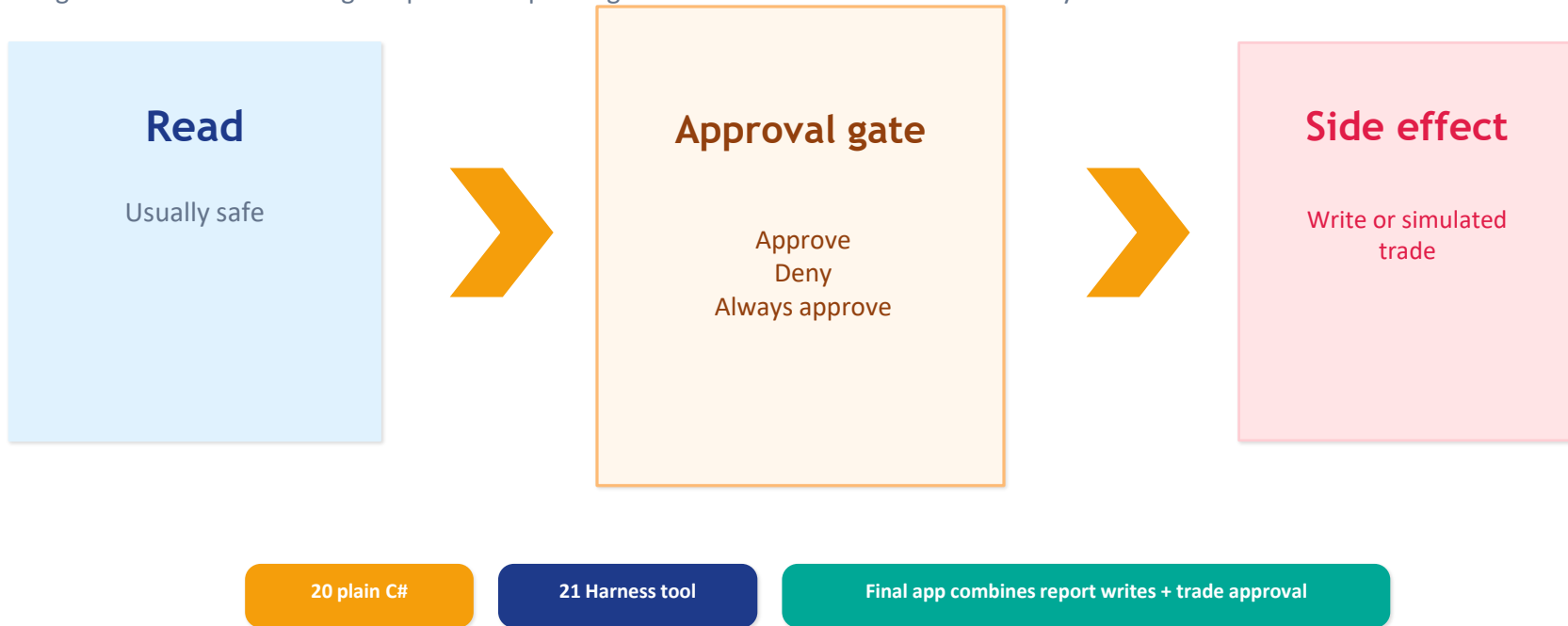
Harness tool:
read_portfolio_summary

Speaker point

The file path boundary is enforced by the app, not by model promises.

Approvals: safe actions need human permission

Reading a file is low risk. Writing a report or requesting a trade crosses a side-effect boundary.



Memory: two kinds, two jobs

Continuity should be explicit enough to debug and durable enough to be useful.

File memory

Curated artifacts the agent stores deliberately: watchlists, notes, and reports under the working folder.

Foundry memory

Durable user facts and preferences managed by the platform when that expansion is enabled.

30 plain C#

31 Harness tools

Final app: `remember_user_preference` + `get_memory`

The Session 2 sample ladder

Every concept has a base demo, an agentic bridge, and then the complete finance advisor.

10	Safe file access	plain C# boundary
11	Safe file access agent	read_portfolio_summary
20	Approval gates	plain C# confirmation
21	Approval gates agent	request_simulated_trade
30	Memory store	plain local JSON memory
31	Memory store agent	remember_user_preference + get_memory

Then: `code\MafClaw.Session02.csproj`

The complete Harness finance advisor combines files, approvals, report writing, and memory.

Live flow for the full app

A short prompt sequence shows that the same boundaries compose into one useful assistant.

```
What is in my portfolio?
```

```
Write a short markdown report.
```

```
Remember that I am a conservative investor.
```

```
Buy 10 shares of MSFT.
```

Tool 1

```
read_portfolio_summary
```

Tool 2

```
write_portfolio_report
```

Tool 3

```
remember_user_preference  
get_memory
```

Tool 4

```
request_simulated_trade
```

What the code proves

The model is not the control plane. The application is.

```
AIPProjectClient(...)
    .GetProjectOpenAIClient()
    .GetResponsesClient()
    .AsIChatClient(model);

chatClient.AsHarnessAgent(new()
{
    ChatOptions = new()
    {
        Tools = [ ... ]
    }
});
```

Host app owns

Allowed paths and working folder

Host app owns

Approval prompts before side effects

Host app owns

Memory keys and persistence model

Model owns

Choosing the right tool for the user request

Recap

Useful

The agent can read files, write reports, and remember context.

Bounded

The app decides which paths, tools, and actions are allowed.

Teachable

Base samples, agentic samples, then one complete finance advisor.

Next: Session 3 expands orchestration, skills, shell access, and background work.